

Course Management & Registration System

A full-stack web application for university course registration

Your Full Name • Student ID • Course Name • Lecturer's Name • October 5, 2026

1. Introduction

Every semester, thousands of university students rush to register for their courses at the same moment the registration window opens. Anyone who has gone through this knows how stressful it can be: classes fill up within minutes, timetables clash, and it is often unclear whether a registration actually went through. On the other side, the academic office has to manage courses, semesters, class sections and rooms, frequently with spreadsheets and manual checks.

In this report, I describe a project that I have worked on to address this problem: the **Course Management & Registration System**. It is a full-stack web application with two portals, one for *students* and one for *administrators*. Students can browse open class sections, register or cancel, see their weekly timetable and look up their grades. Administrators can manage courses, semesters, class sections and schedules, and monitor registrations in real time.

2. Background and Motivation

2.1 The problem

Course registration looks simple, but it hides several hard problems:

- **Capacity:** each class section has a fixed number of seats, and many students may try to take the last seat at the same time.
- **Timetable conflicts:** a student must not register for two sections that meet at the same time.
- **Time windows:** registration and cancellation are only allowed during a period set by the academic office.
- **Account status:** suspended students must not be able to register.
- **Traceability:** cancelled registrations should be kept for auditing instead of being deleted.

2.2 Objectives

My objectives for the project were to:

1. build a working web application with separate student and admin portals;
2. enforce all registration rules on the server so that the data always stays consistent, even under concurrent requests;
3. design a clean, normalised relational database;
4. make the system easy to deploy with a single command using Docker;
5. practise professional development habits: a layered architecture, documentation and version control with Git.

3. Main Features

The system has two portals. A user logs in once, and the JSON Web Token they receive tells the application which portal to open.

Student portal	
Student information	View personal profile, major, academic year and status.
Course registration	Browse open sections, register with one click, cancel a registration; conflicts are reported immediately.
Timetable	Weekly timetable built from the student's registered sections (supports odd/even weeks).
Grade lookup	Midterm, final and total scores, letter grades and GPA.
Administrator portal	
Course management	Create, edit and deactivate courses; set credits and prerequisites.
Semester management	Define semesters, start/end dates and the registration window; mark the current semester.
Class section management	Open sections for a course, assign lecturer, room and capacity.
Schedule management	Define day of week, periods, room and week type for each section.
Registration tracking	Monitor all enrollments and section fill rates.

Table 1: Main features of the two portals

The user interface was built with my own CSS in a “glassmorphism” style and includes a **dark mode**, implemented with a React context so that the theme choice applies across all pages.

4. System Design

4.1 Technology stack

Layer	Technologies
Frontend	React 19, Vite, React Router, custom CSS (glassmorphism, dark mode)
Backend	Node.js 18+, Express.js 4, Sequelize 6 ORM, express-validator
Database	MySQL 8.0 (utf8mb4 for full Vietnamese text support)
Security	JWT authentication, bcrypt password hashing, role-based access control
DevOps	Docker & Docker Compose, Nginx (serves the built frontend), npm workspaces (monorepo), Makefile

Table 2: Technology stack

4.2 Architecture

The project is organised as a **monorepo** with two workspaces, **client** and **server**. The backend follows a layered architecture, where each layer has a single responsibility (Figure 1):

- **Routes** map URLs such as `POST /api/enrollments` to controllers.
- **Middleware** handles authentication (JWT), authorisation (student/admin roles), input validation and errors.

- **Controllers** read the request and send the response.
- **Services** contain the business logic, for example the enrollment and schedule-conflict rules.
- **Models** (Sequelize) map JavaScript objects to MySQL tables.

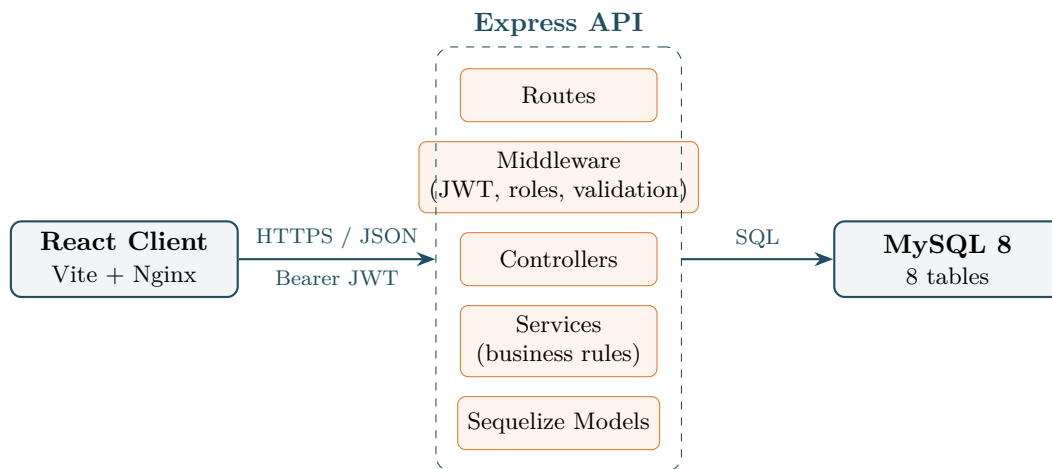


Figure 1: Overall architecture. All three parts run as Docker containers.

Every API response uses the same JSON format (`success`, `message`, `data`), and errors are thrown as a custom `ApiError` class with an HTTP status code, so the frontend can handle all errors in one place.

4.3 Database design

The database has eight tables. Table 3 summarises them and Figure 2 shows the main relationships.

Table	Purpose
<code>students</code>	Student accounts and profiles (ID, major, academic year, status)
<code>admins</code>	Administrator accounts
<code>courses</code>	Course catalogue with credits and an optional prerequisite (self-reference)
<code>semesters</code>	Semesters with dates and the registration window (<code>reg_open</code> , <code>reg_close</code>)
<code>sections</code>	Class sections of a course in a semester, with lecturer, room and capacity
<code>schedules</code>	Meeting times of a section: day, start/end period, room, week type
<code>enrollments</code>	Which student registered for which section, and the status
<code>results</code>	Midterm, final and total scores and letter grades

Table 3: Database tables

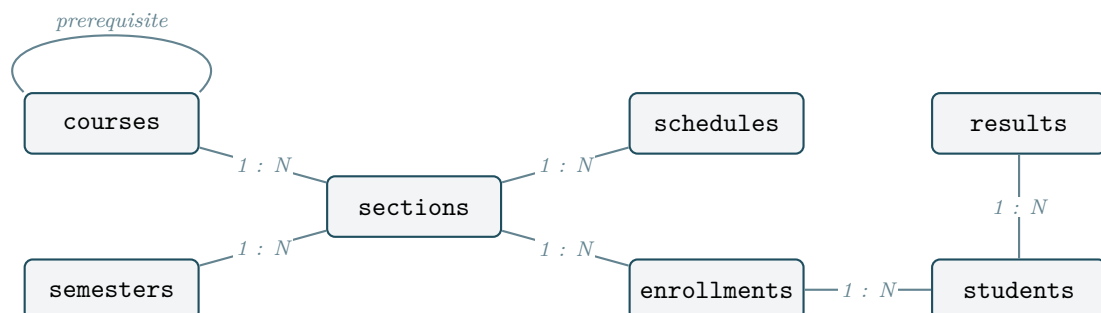


Figure 2: Simplified entity-relationship diagram

Important rules are also enforced inside the database, as a second line of defence:

- a **unique index** on (`student_id`, `section_id`) in `enrollments` prevents duplicate registrations;

- a **CHECK constraint** guarantees `current_students <= max_students`;
- scores must be between 0 and 10, and `day_of_week` must be valid;
- indexes on foreign keys and status columns keep common queries fast.

5. Registration Business Rules

The heart of the project is the enrollment service. I wrote down nine business rules (BR01–BR09) before coding and then implemented each one explicitly.

Rule	Description
BR01	A student cannot register for the same section twice.
BR02	Registration is refused when the section is full.
BR03	Registration is only allowed inside the semester's registration window.
BR04	The system detects and blocks timetable conflicts.
BR05	Suspended students cannot register.
BR06	Closed or cancelled sections cannot be registered.
BR07	Cancellation is only allowed inside the registration window.
BR08	The section's student counter decreases automatically on cancellation.
BR09	Cancellations are <i>soft deletes</i> : the record is kept with status <code>cancelled</code> .

Table 4: Business rules for registration and cancellation

5.1 Preventing race conditions

The most difficult rule was BR02. If two students try to take the last seat at the same moment, a naive implementation reads “1 seat left” for both and registers both, so the class becomes over-full. To prevent this, the whole registration runs inside a **database transaction**, and the section row is locked with `SELECT ... FOR UPDATE`. The second request has to wait until the first one commits, and then it sees that the section is full.

```
const transaction = await sequelize.transaction();
try {
  // Row-level lock: other requests for this section must wait
  const section = await Section.findByPk(sectionId, {
    include: [{ model: Semester, as: 'semester' }],
    lock: transaction.LOCK.UPDATE,
    transaction,
  });
  if (section.current_students >= section.max_students) {
    throw ApiError.badRequest('Section_is_full'); // BR02
  }
  // ... BR01, BR03-BR06 checks ...
  await Enrollment.create({ student_id, section_id, status: 'enrolled' }, { transaction });
  await Section.increment('current_students', { by: 1, where: { section_id }, transaction });
  await transaction.commit();
} catch (error) {
  await transaction.rollback(); // all-or-nothing
  throw error;
}
```

5.2 Detecting timetable conflicts

For BR04, the schedule service compares every meeting time of the new section with the meeting times of all sections the student is already enrolled in for the same semester. Two meetings conflict when they are on the same day, their period ranges overlap, and their week types are compatible (a class held only in odd weeks never clashes with one held only in even weeks):

```
_isConflict(a, b) {  
  if (a.day_of_week !== b.day_of_week) return false;  
  // [startA, endA] ∩ [startB, endB] ≠ ∅  
  const overlap = a.start_period <= b.end_period && a.end_period >= b.start_period;  
  if (!overlap) return false;  
  if (a.week_type === 'odd' && b.week_type === 'even') return false;  
  if (a.week_type === 'even' && b.week_type === 'odd') return false;  
  return true;  
}
```

When a conflict is found, the API returns HTTP 409 **Conflict** together with the details of the clashing section, so the student knows exactly which class to change.

6. Development Process

I developed the project between January and March 2026 and used Git throughout. The work went through five main stages:

1. **Analysis (January).** I studied how registration works at my university, listed the features for each type of user and wrote the nine business rules.
2. **Database and backend (March).** I designed the eight-table schema, wrote the SQL migration and seed data, and implemented the Express API: authentication, courses, semesters, sections, schedules, enrollments and results.
3. **Restructuring.** When the project grew, I reorganised it into a monorepo with `client` and `server` workspaces, so both can be started with one command (`npm run dev`).
4. **Frontend.** I built the student site and the admin site in React, then refined the UI and added dark mode.
5. **Deployment and documentation.** I added Dockerfiles and a Docker Compose file that starts MySQL, the API and the Nginx frontend together, with health checks. I also wrote documentation for the backend API, the database and Docker deployment.

7. Challenges and Solutions

Challenge	Solution
Over-booking when many students register at once	Database transactions with row-level locking, atomic counter updates and a CHECK constraint as a safety net.
Correct timetable-conflict logic, including odd/even weeks	A small, pure function <code>_isConflict</code> that is easy to reason about and test.
Keeping history of cancellations	Soft delete with a <code>status</code> field, <code>cancelled_at</code> and <code>cancel_reason</code> ; re-registration reuses the old record.
Securing two kinds of users	JWT with a role claim, plus <code>authorizeAdmin</code> , <code>authorizeStudent</code> and <code>authorizeSelf</code> middleware.
“It works on my machine”	Docker Compose with environment files for development, Docker and production.

Table 5: Main challenges and how I solved them

8. What I Learned

This project taught me much more than any single course exercise:

- **Technical skills.** I became confident with React, Express, Sequelize and MySQL, and I now understand why transactions and locks matter in real systems.
- **Thinking before coding.** Writing the business rules first made the implementation much clearer, and each rule maps to a few lines of code that are easy to find.
- **Clean architecture.** Separating routes, controllers, services and models made it easy to change one part without breaking others.
- **Professional habits.** Using Git, writing documentation and containerising the application are habits I will keep in future projects.

8.1 Future improvements

Next, I would like to enforce course prerequisites automatically, add a full automated test suite with Jest, offer a waiting list with email notifications for full sections, and add caching and wider rate limiting (today only the login route is limited) to handle the peak load when registration opens.

9. Conclusion

The Course Management & Registration System is the project I am most proud of so far. It solves a real problem that every student experiences, and it works from end to end: from the React interface, through a secure Express API, to a carefully designed MySQL database, all packaged with Docker. Most importantly, building it taught me how to turn a set of real-world rules into reliable software, and how to organise a project so that it can keep growing.